

Reporte de Práctica 02: Simulador de Estacionamiento

Universidad Autónoma de Baja California
Facultad de Ingeniería, Arquitectura y Diseño

Campo	Detalle
Materia	40032 - Paradigmas de la Programación
Docente	M.I. José Carlos Gallegos Mariscal
Grupo	942
Entregable	PP_PL2_Apellido_Matricula

1. Introducción

Problema

El sistema desarrollado administra un estacionamiento con lugares (*spots*), vehículos y tickets. El problema central consiste en gestionar la entrada y salida de vehículos de forma ordenada: asignar espacios disponibles compatibles, registrar el tiempo de permanencia, calcular el costo de acuerdo con una política de cobro y mostrar en todo momento el estado de ocupación del estacionamiento.

Las reglas fundamentales del sistema son:

- Un lugar solo puede estar ocupado por un vehículo a la vez.
- Un ticket activo representa un vehículo dentro del estacionamiento.
- La entrada asigna un lugar compatible y crea un ticket activo.
- La salida cierra el ticket, libera el lugar y calcula el costo.
- Si no hay lugares disponibles compatibles, se rechaza la entrada.

Objetivos

1. Diseñar e implementar un sistema completo aplicando **Programación Orientada a Objetos (POO)**: modelo, encapsulación, abstracción, herencia, composición, polimorfismo y subtipos.
 2. Construir un prototipo funcional en consola con las operaciones mínimas: registrar entrada, registrar salida, ver ocupación y ver tickets activos.
 3. Aplicar polimorfismo mediante políticas de cobro intercambiables (**RatePolicy**) y subtipos de vehículo (**Car**, **Motorcycle**).
 4. Integrar el modelo en una interfaz web con **Flask** bajo el patrón **MVC**.
 5. Elaborar un reporte técnico con evidencias de código y ejecución.
-

2. Modelo del dominio

Diagrama UML

El diagrama incluye las relaciones de herencia entre `Vehicle`, `Car` y `Motorcycle`; la composición entre `ParkingLot`, `ParkingSpot` y `Ticket`; y la dependencia de `ParkingLot` hacia la interfaz `RatePolicy`.

Lista de clases y responsabilidades

Clase	Responsabilidad
<code>Vehicle</code>	Abstracción base del vehículo; contiene placas y tipo
<code>Car</code>	Subtipo de <code>Vehicle</code> que representa un automóvil
<code>Motorcycle</code>	Subtipo de <code>Vehicle</code> que representa una motocicleta
<code>ParkingSpot</code>	Representa un espacio individual; conoce su tipo permitido y su estado (libre/ocupado)
<code>Ticket</code>	Registra id, vehículo, spot asignado, hora de entrada, hora de salida y estado (activo/cerrado)
<code>ParkingLot</code>	Administra la colección de spots, los tickets activos y el total recaudado; coordina entrada y salida
<code>RatePolicy</code>	Interfaz abstracta que define el contrato <code>calculate(hours, vehicle)</code> para calcular el costo
<code>HourlyRatePolicy</code>	Implementación de <code>RatePolicy</code> ; cobra por hora transcurrida
<code>FlatRatePolicy</code>	Implementación de <code>RatePolicy</code> ; cobra una tarifa fija independiente del tiempo

3. Evidencia de conceptos POO (*obligatorio*)

3.1 Encapsulación

Método que valida y protege el estado interno — ningún spot puede quedar con dos vehículos al mismo tiempo.

```
from dataclasses import dataclass, field

@dataclass
class ParkingSpot:
    spot_id: str
    allowed_type: str
    __is_occupied: bool = field(default=False, init=False, repr=False)

    def occupy(self) -> None:
        """Valida que el spot esté libre antes de ocuparlo."""
        if self.__is_occupied:
            raise ValueError(f"El lugar {self.spot_id} ya está ocupado.")
        self.__is_occupied = True

    def release(self) -> None:
        """Valida que el spot esté ocupado antes de liberarlo."""
```

```

    if not self.__is_occupied:
        raise ValueError(f"El lugar {self.spot_id} ya está libre.")
    self.__is_occupied = False

def is_occupied(self) -> bool:
    return self.__is_occupied

def accepts(self, vehicle_type: str) -> bool:
    """Protege la regla de compatibilidad de tipo."""
    return self.allowed_type in ("Any", vehicle_type)

```

3.2 Abstracción

Interfaz `RatePolicy` que desacopla la lógica de cobro del resto del sistema.

```

# models/rates.py
from typing import Protocol

class RatePolicy(Protocol):
    """Contrato común para cualquier política de cobro."""
    def calculate(self, hours: float, vehicle_type: str) -> float:
        ...

class HourlyRatePolicy:
    """Cobra $25 por hora, con descuento del 20 % para motos."""
    def calculate(self, hours: float, vehicle_type: str) -> float:
        base = hours * 25.0
        return base * 0.8 if vehicle_type == "Motorcycle" else base

class FlatRatePolicy:
    """Tarifa fija de $80, independiente del tiempo."""
    def calculate(self, hours: float, vehicle_type: str) -> float:
        return 80.0

```

3.3 Composición

`ParkingLot` administra colecciones de `ParkingSpot` y `Ticket`; la política de cobro se inyecta como dependencia.

```

# models/parking_lot.py
from models.spot import ParkingSpot
from models.ticket import Ticket
from models.rates import RatePolicy

class ParkingLot:
    def __init__(self, spots: list[ParkingSpot], rate_policy: RatePolicy):
        self.__spots: list[ParkingSpot] = spots          # composición

```

```

        self.__tickets: dict[int, Ticket] = {}
        self.__rate_policy: RatePolicy = rate_policy # inyección de
dependencia
        self.__next_ticket_id: int = 1
        self.__total_collected: float = 0.0

    def park(self, vehicle) -> Ticket:
        spot = self.__find_available_spot(vehicle.vehicle_type)
        if spot is None:
            raise Exception("No hay lugares disponibles compatibles.")
        spot.occupy()
        ticket = Ticket(self.__next_ticket_id, vehicle, spot)
        self.__tickets[ticket.ticket_id] = ticket
        self.__next_ticket_id += 1
        return ticket

    def checkout(self, ticket_id: int, hours: float) -> float:
        ticket = self.__tickets.get(ticket_id)
        if ticket is None or not ticket.is_active():
            raise ValueError(f"Ticket #{ticket_id} no existe o ya fue cerrado.")
        fee = self.__rate_policy.calculate(hours, ticket.vehicle.vehicle_type)
        ticket.close(hours, fee)
        ticket.spot.release()
        self.__total_collected += fee
        return fee

    def __find_available_spot(self, vehicle_type: str):
        return next(
            (s for s in self.__spots if not s.is_occupied() and
s.accepts(vehicle_type)),
            None
        )

```

3.4 Herencia / Subtipos

Clase base `Vehicle` con subclases `Car` y `Motorcycle`.

```

# models/vehicle.py
from dataclasses import dataclass

@dataclass
class Vehicle:
    plate: str
    vehicle_type: str = "Vehicle" # se sobreescribe en subclases

    def describe(self) -> str:
        return f"{self.vehicle_type} | Placas: {self.plate}"

@dataclass
class Car(Vehicle):

```

```

    vehicle_type: str = "Car"

@dataclass
class Motorcycle(Vehicle):
    vehicle_type: str = "Motorcycle"

```

3.5 Polimorfismo

La interfaz común `RatePolicy` permite cambiar el comportamiento de cobro sin modificar `ParkingLot`.

```

from models.parking_lot import ParkingLot
from models.rates import HourlyRatePolicy, FlatRatePolicy
from models.spot import ParkingSpot
from models.vehicle import Car, Motorcycle

spots = [ParkingSpot("A1", "Car"), ParkingSpot("M1", "Motorcycle")]

# Estacionamiento con tarifa por hora
lot_hourly = ParkingLot(spots[:], HourlyRatePolicy())

# Estacionamiento con tarifa plana
lot_flat = ParkingLot(spots[:], FlatRatePolicy())

car = Car(plate="ABC-123")
moto = Motorcycle(plate="XYZ-777")

t1 = lot_hourly.park(car)
print(lot_hourly.checkout(t1.ticket_id, hours=2))    # → $50.0  (2h × $25)

t2 = lot_flat.park(moto)
print(lot_flat.checkout(t2.ticket_id, hours=5))      # → $80.0  (tarifa fija)

```

El mismo método `checkout` produce resultados distintos dependiendo de la política asignada al estacionamiento — **sin cambiar una sola línea de `ParkingLot`**.

4. MVC con Flask

Separación de capas

Capa	Componente	Descripción
Model	<code>models/</code>	Clases <code>ParkingLot</code> , <code>ParkingSpot</code> , <code>Vehicle</code> , <code>Car</code> , <code>Motorcycle</code> , <code>Ticket</code> , <code>RatePolicy</code> , <code>HourlyRatePolicy</code> , <code>FlatRatePolicy</code> . Contienen toda la lógica de negocio.

Capa	Componente	Descripción
View	templates/	Plantillas HTML con Jinja2: <code>base.html</code> , <code>dashboard.html</code> , <code>entry.html</code> , <code>exit.html</code> . Solo presentan datos, no deciden nada.
Controller	app.py	Rutas Flask que reciben peticiones HTTP, delegan al Model y devuelven la View correspondiente. La lógica de negocio no se escribe aquí.

Estructura de archivos

```
parking_lot/
├── app.py                ← Controller (rutas Flask)
├── models/
│   ├── vehicle.py
│   ├── spot.py
│   ├── ticket.py
│   ├── parking_lot.py
│   └── rates.py
├── templates/
│   ├── base.html
│   ├── dashboard.html ← ocupación + tickets activos
│   ├── entry.html     ← formulario de entrada
│   └── exit.html       ← formulario de salida
└── static/
    └── styles.css
```

Rutas definidas

Método	Ruta	Pantalla / Acción
GET	/	Dashboard — ocupación y tickets activos
GET	/entry	Formulario de registro de entrada
POST	/entry	Procesa la entrada y redirige al dashboard
GET	/exit	Formulario de registro de salida
POST	/exit	Procesa la salida, muestra el cobro

5. Pruebas manuales

Flujo 1 — Entrada y salida exitosa de un automóvil

```
Entrada: placas=ABC-123  tipo=Car      → Ticket #1, spot=A1
Salida:  ticket=1        horas=2       → costo=$50.00, spot liberado=A1
```

Paso	Acción	Resultado esperado	Resultado obtenido
1	Ingresar placas ABC-123 , tipo Car en /entry	Ticket #1 generado, spot A1 ocupado	✓
2	Ver dashboard /	A1 aparece como ocupado, 1 ticket activo	✓
3	Registrar salida ticket #1 con 2 horas en /exit	Costo mostrado: \$50.00	✓
4	Ver dashboard /	A1 liberado, 0 tickets activos	✓

Flujo 2 — Entrada simultánea de dos vehículos y rechazo por capacidad

```
Entrada: placas=ABC-123  tipo=Car          → Ticket #1, spot=A1
Entrada: placas=XYZ-777  tipo=Motorcycle → Ticket #2, spot=M1
Ocupación: libres=0  ocupados=2
Entrada: placas=ERR-000  tipo=Car          → RECHAZADO (sin lugares disponibles)
```

Paso	Acción	Resultado esperado	Resultado obtenido
1	Registrar entrada ABC-123 (Car)	Ticket #1, spot A1	✓
2	Registrar entrada XYZ-777 (Motorcycle)	Ticket #2, spot M1	✓
3	Ver dashboard	0 lugares libres, 2 tickets activos	✓
4	Intentar registrar ERR-000 (Car)	Mensaje de error: "No hay lugares disponibles"	✓
5	Registrar salida ticket #1	Spot A1 liberado, costo calculado	✓
6	Reintentar entrada ERR-000 (Car)	Ticket #3, spot A1 asignado correctamente	✓

6. Conclusiones

- La aplicación del patrón **MVC** con Flask permitió una separación clara entre la lógica de negocio y la presentación. Las rutas en **app.py** actúan exclusivamente como coordinadores, sin contener reglas del dominio.
- Los principios de **encapsulación** y **composición** quedaron evidenciados en **ParkingLot** y **ParkingSpot**: ningún atributo interno es modificable directamente desde fuera, y las invariantes del sistema (un spot, un vehículo a la vez) se garantizan desde los propios métodos.
- El uso de **Protocol** de Python para definir **RatePolicy** demostró cómo el **polimorfismo** permite extender el comportamiento del sistema (nuevas tarifas) sin modificar las clases existentes, cumpliendo

el principio Open/Closed.

- La **inyección de dependencia** de la política de cobro en **ParkingLot** facilitó las pruebas y hace que cambiar la tarifa en producción sea tan simple como pasar una instancia distinta.
- El uso de **dataclasses** redujo significativamente el código repetitivo en las entidades del dominio, manteniendo el código limpio y legible.

7. Referencias en formato APA

Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection pattern*.

<https://martinfowler.com/articles/injection.html>

Pallets Projects. (2026). *Welcome to Flask — Flask Documentation (3.1.x)*. <https://flask.palletsprojects.com/>

Pallets Projects. (2026). *Quickstart — Flask Documentation (3.1.x)*.

<https://flask.palletsprojects.com/en/stable/quickstart/>

Python Software Foundation. (2026). *dataclasses — Data Classes*.

<https://docs.python.org/3/library/dataclasses.html>

Python Typing Team. (2026). *Protocols — typing specification*.

<https://typing.python.org/en/latest/spec/protocol.html>

Flask-es Read the Docs. (2026). *Plantillas — Documentación de Flask (Tutorial)*. [https://flask-](https://flask-es.readthedocs.io/tutorial/templates/)

[es.readthedocs.io/tutorial/templates/](https://flask-es.readthedocs.io/tutorial/templates/)
